

Αλγοριθμικές Τεχνικές για Δεδομένα Ευρείας Κλίμακας

1° Φυλλάδιο Ασκήσεων

Προθεσμία υποβολής: 07/04/2026, 23:55

Ενότητα Α: Μετρητές Morris (Μονάδες: 4.5)

Θεωρούμε έναν μετρητή τύπου Morris που εκτιμά το πλήθος στοιχείων n σε μια ροή δεδομένων, ο οποίος περιγράφεται από τον εξής ψευδοκώδικα:

integer $C \leftarrow 0$	
insert()	query()
{	{
$C \leftarrow C+1$, with probability $1/\alpha^C$	return $(1/(\alpha-1)) \cdot (\alpha^C - 1)$
}	}

όπου το “ α ” είναι ένας πραγματικός αριθμός μεγαλύτερος του 1 που καθορίζεται από τον χρήστη. (Η παράμετρος “ α ” καθορίζει την ακρίβεια του μετρητή: όσο πιο κοντά βρίσκεται στο 1, τόσο καλύτερη αναμένεται να είναι η εκτίμηση του n που δίνει ο μετρητής.)

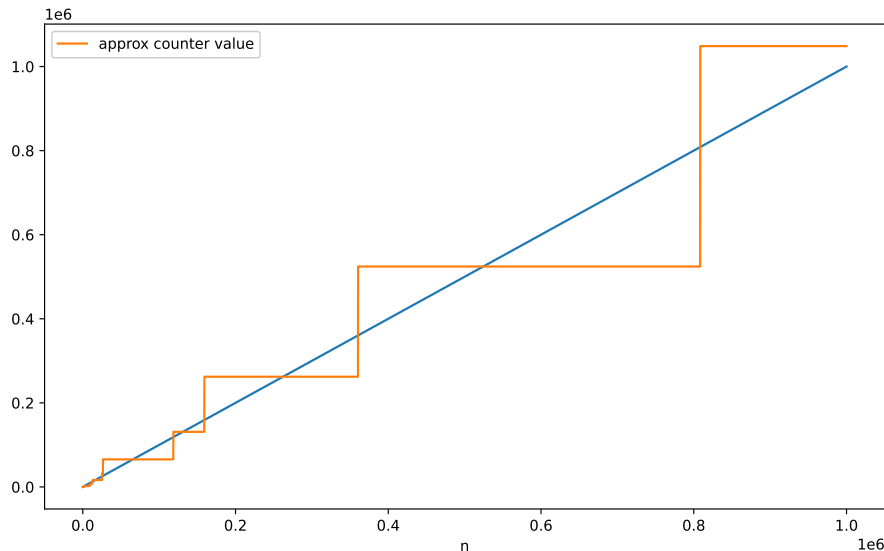
Για κάθε χρήση του μετρητή Morris για n εισαγωγές στοιχείων, θεωρούμε ότι η κατανάλωση μνήμης του είναι το πλήθος των bits που έχει καταλάβει η μεταβλητή C . (Επομένως, η κατανάλωση μνήμης είναι $\lceil \log_2(C+1) \rceil$.)

Άσκηση 1

α) Υλοποιήστε τον παραπάνω μετρητή Morris για $\alpha=2$, και τρέξτε τον για 1.000.000 εισαγωγές στοιχείων. Έπειτα από κάθε εισαγωγή, εκτυπώστε την εκτίμηση του μετρητή (δηλαδή το $2^C - 1$).

Σχεδιάστε μία αντίστοιχη γραφική παράσταση που δείχνει τις τιμές του μετρητή καθώς γίνοντουσαν οι εισαγωγές. Το αποτέλεσμα σας θα πρέπει να είναι όπως φαίνεται στην ακόλουθη εικόνα¹:

¹ Προφανώς η γραφική παράσταση θα ποικίλλει ανάλογα με την αρχικοποίηση που κάνατε στην συνάρτηση που παράγει την τυχαιότητα.



Παράδειγμα εκτιμήσεων του μετρητή Morris για το πλήθος εισαγωγών n , όσο το n αυξάνεται από 1 μέχρι και 1.000.000.

(Μονάδες: 0.5)

β) Μπορούμε να βελτιώσουμε τον παραπάνω μετρητή ως εξής. Αντί να κρατούμε μόνο μία μεταβλητή C , μπορούμε να κρατούμε αρκετές τέτοιες μεταβλητές (οι οποίες ανανεώνονται ανεξάρτητα), και να επιστρέφουμε τον μέσο όρο ή τον διάμεσο από το αντίστοιχο αποτέλεσμα που δίνουν. (Π.χ., αν κρατούμε τις μεταβλητές C_1, C_2, C_3 , τότε επιστρέφουμε ως εκτίμηση του n το $(2^{C_1} + 2^{C_2} + 2^{C_3} - 3)/3$.)

Υλοποιήστε αυτήν την ιδέα, για τον μέσο όρο και τον διάμεσο, χρησιμοποιώντας 5 ανεξάρτητες μεταβλητές, και κάντε τις αντίστοιχες γραφικές παραστάσεις όπως και στο α). Ποια από τις δύο αυτές επιλογές (δηλαδή διάμεσο και μέσο όρο) θεωρείτε ότι δίνει καλύτερα αποτελέσματα;

(Μονάδες: 0.5)

γ) Προφανώς είναι ανούσιο στην πράξη να κρατά κανείς πέντε διαφορετικές μεταβλητές C για να εκτιμήσει το πλήθος στοιχείων όταν το n φτάνει μέχρι το 1.000.000. Πρώτον, τεκμηριώστε αυτόν τον ισχυρισμό. Δεύτερον, για ποια n θα είχε νόημα να κρατά κανείς 5 μεταβλητές C , δεδομένου ότι θέλουμε να έχουμε κέρδος στον χώρο; (Θεωρήστε ότι μόνο οι μεταβλητές C είναι αυτές που κοστίζουν χώρο μνήμης.)

(Μονάδες: 0.25)

δ) Έστω ότι έχουμε διαθέσιμα 8 bits μνήμης και θέλουμε να εκτιμήσουμε το πλήθος στοιχείων μετά από 1.000.000 εισαγωγές. Παρατηρήστε ότι τα διαθέσιμα bitάκια είναι περισσότερα απ' όσα χρειαζόμαστε αν θέσουμε $\alpha=2$ (γιατί;). Αν όμως θέσουμε το α να είναι πολύ κοντά στο 1, τότε ο εσωτερικός μετρητής C θα αυξάνεται πολύ πιο γρήγορα απ' ό,τι αν έχουμε $\alpha=2$, με αποτέλεσμα να σκαλώσει στον αριθμό $2^8 - 1 = 127$, και άρα η εκτίμηση που θα επιστρέφει θα είναι συνήθως μία υποεκτίμηση του 1.000.000 (διότι επιστρέφεται ως εκτίμηση το $(1/(\alpha-1)) \cdot (\alpha^C - 1)$).

Δοκιμάστε να κάνετε ορισμένες γραφικές παραστάσεις όπως και στο α) αυτής της άσκησης, για διάφορες επιλογές του α . Βρείτε ένα α το οποίο, κατά την γνώμη σας,

δίνει έναν μετρητή Morris με όσο το δυνατόν καλύτερη απόδοση, υπό τον περιορισμό των 8 bits για την εσωτερική μεταβλητή C.

(Μονάδες: 0.5)

ε) Έστω ότι έχουμε διαθέσιμα 15 bits μνήμης. Προτείνουμε δύο διαφορετικούς αλγορίθμους για εκτίμηση του πλήθους στοιχείων όταν $n=1.000.000$.

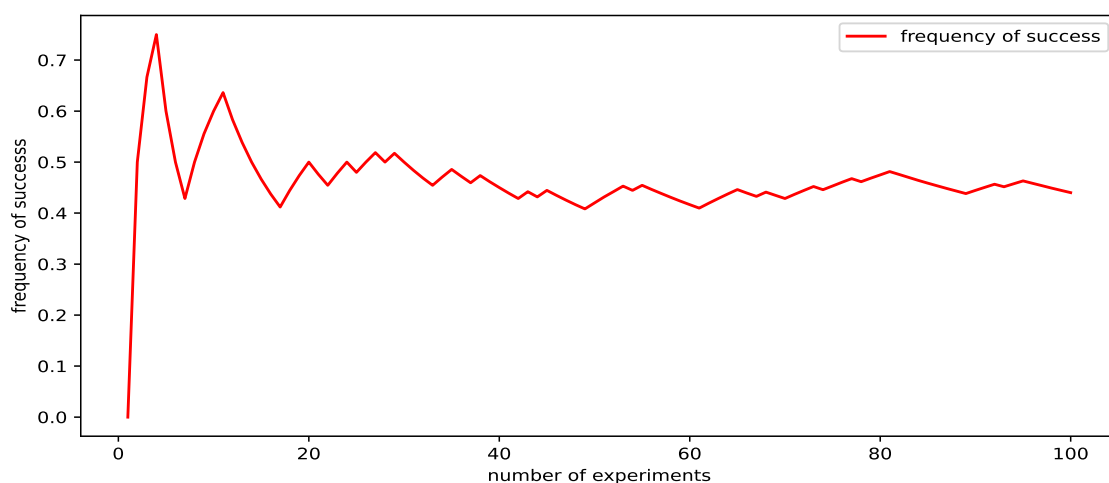
Ο πρώτος αλγόριθμος χρησιμοποιεί έναν μετρητή τύπου Morris με $\alpha=2$.

Ο δεύτερος αλγόριθμος χρησιμοποιεί τρεις μετρητές τύπου Morris, όπου το α θα καθοριστεί από εσάς, και επιστρέφει τον μέσο όρο τους. Ο κάθε ένας από τους τρεις μετρητές θα χρησιμοποιεί 5 από τα 15 διαθέσιμα bitάκια μνήμης. (Επομένως, η εσωτερική μεταβλητή C του καθενός δεν θα πρέπει να ξεπεράσει το 31. Δηλαδή, αν τύχει να φτάσει στο 31, θα πρέπει να παραμείνει εκεί.)

Για κάθε έναν από τους δύο αυτούς αλγορίθμους, με χρήση της τεχνικής Monte Carlo εκτιμήστε την πιθανότητα η απάντηση μετά από 1.000.000 εισαγωγές στοιχείων να είναι το λιγότερο 800.000 και το πολύ 1.200.000. Για τον δεύτερο αλγόριθμο, κάντε μία τέτοια επιλογή του α , ώστε, σύμφωνα με τα πειράματα Monte Carlo, να έχουμε την ένδειξη ότι έχουμε καλύτερη πιθανότητα επιτυχίας σε σχέση με τον πρώτο.

Επιπλέον, σχεδιάστε και τις αντίστοιχες γραφικές παραστάσεις που δείχνουν το ποσοστό των επιτυχιών μετά από κάθε πλήθος πειραμάτων. (Δηλαδή ο άξονας των x δείχνει το πλήθος των πειραμάτων που έγιναν, και ο άξονας των y δείχνει το ποσοστό των επιτυχιών. Πείραμα θεωρείται κάθε τρέξιμο του αλγορίθμου για 1.000.000 εισαγωγές στοιχείων.)

Για παράδειγμα, για τον πρώτο αλγόριθμο η γραφική παράσταση με 100 πειράματα μπορεί να μοιάζει κάπως έτσι (οπότε εκτιμούμε ότι η πιθανότητα επιτυχίας είναι περίπου 45%, αν και μάλλον χρειαζόμαστε περισσότερα πειράματα για μεγαλύτερη ακρίβεια):



(Μονάδες: 0.75)

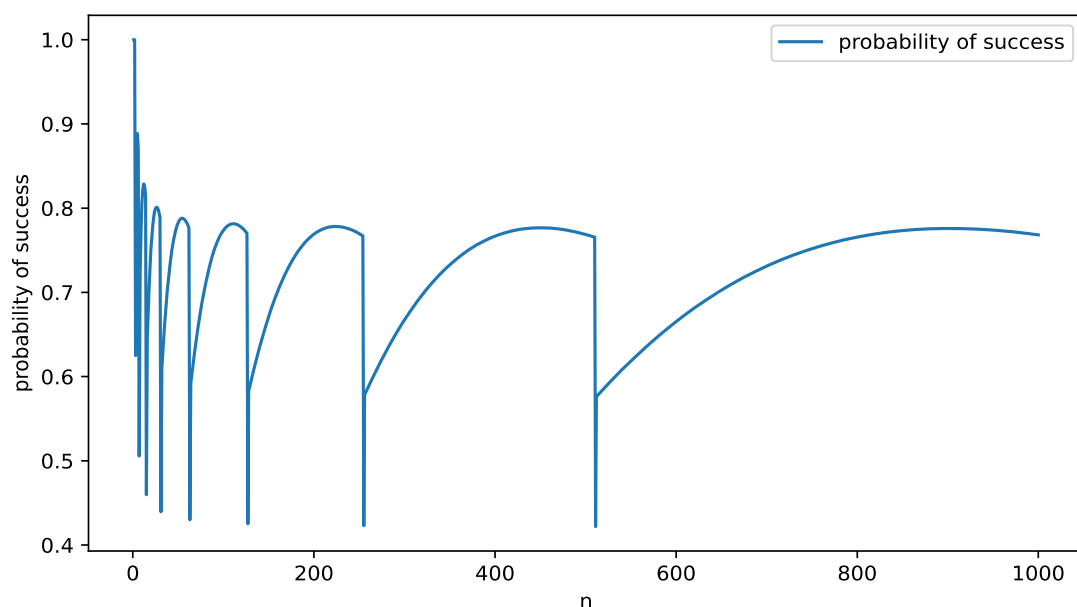
Άσκηση 2

α) Πρόκειται να δεχτούμε το πολύ 1000 εισαγωγές στοιχείων, και με χρήση ενός μετρητή Morris με $\alpha=2$ θέλουμε να εκτιμήσουμε το πλήθος στοιχείων που είδαμε. Υπολογίστε την μικρότερη πιθανότητα επιτυχίας που μπορεί να έχουμε, αν θέλουμε η εκτίμηση να βρίσκεται μεταξύ n/c και cn , όπου n είναι το πλήθος εισαγωγών στοιχείων (δηλαδή $n \in \{1, 2, \dots, 1000\}$), και $c \in \{2, 3, 4, 5\}$. Με άλλα λόγια, αν συμβολίσουμε με E_n την εκτίμηση του μετρητή μετά από n εισαγωγές στοιχείων, τότε θέλουμε να υπολογίσουμε την ποσότητα:

$$\min\{P(E_n \geq n/c \text{ και } E_n \leq cn) \mid n \in \{1, 2, \dots, 1000\}\}.$$

Θέλουμε αυτός ο υπολογισμός να γίνει με απόλυτη ακρίβεια στα πρώτα 10 δεκαδικά ψηφία. (Για τον σκοπό αυτόν, προτείνεται να χρησιμοποιήσετε την βιβλιοθήκη Fraction της Python, για χειρισμό κλασμάτων με μεγάλους αριθμητές και παρονομαστές.)

Επιπλέον, σχεδιάστε και τις γραφικές παραστάσεις όλων των πιθανοτήτων επιτυχίας



για όλα τα n μεταξύ 1 και 1000, και για όλα τα $c \in \{2, 3, 4, 5\}$. Επομένως, για $c=2$, η γραφική παράσταση θα μοιάζει όπως στο παραπάνω σχήμα (και η ελάχιστη ζητούμενη πιθανότητα είναι 0.4218567224, η οποία δίνεται για $n=511$).

(Μονάδες: 1)

β) Στο προηγούμενο πρόβλημα υπολογίσαμε ορισμένα κάτω φράγματα για την πιθανότητα επιτυχίας ενός μετρητή Morris όταν πρόκειται να δεχτούμε το πολύ 1000 εισαγωγές στοιχείων. Συγκεκριμένα, αν π.χ. θεωρήσουμε ως επιτυχία το να έχουμε μία εκτίμηση που είναι το πολύ 2 φορές η πραγματική τιμή και τουλάχιστον το μισό της πραγματικής τιμής, τότε, αν γίνουν n εισαγωγές στοιχείων, με $n \in \{1, 2, \dots, 1000\}$, η πιθανότητα επιτυχίας είναι τουλάχιστον 42%.

Φυσικά, το γεγονός ότι μετά από n εισαγωγές στοιχείων έχουμε πιθανότητα τουλάχιστον 42% η εκτίμηση να βρίσκεται μεταξύ $n/2$ και $2n$, δεν συνεπάγεται ότι

αυτό ίσχυε καθόλη την διάρκεια των εισαγωγών. Με άλλα λόγια, καθώς γινόντουσαν οι εισαγωγές, αν δοκιμάζαμε να επιστρέψουμε την εκτίμηση μετά από κάθε εισαγωγή, πιθανότατα θα είχαμε αρκετές φορές υποεκτίμηση, και αρκετές φορές υπερεκτίμηση. (Αλλά η πιθανότητα επιτυχίας για n εισαγωγές, όπου το n έχει σταθεροποιηθεί εξ αρχής, είναι πάντοτε τουλάχιστον 42%.)

Έστω ότι μας ενδιαφέρει η εκτίμηση να βρίσκεται μεταξύ n/c και cn καθόλη την διάρκεια των εισαγωγών. Τότε, υπολογίστε με απόλυτη ακρίβεια στα πρώτα 10 δεκαδικά ψηφία τις πιθανότητες να είχαμε επιτυχία καθόλη την διάρκεια των 1000 εισαγωγών στοιχείων, για $c \in \{2, 3, 4, 5\}$.

Αν δεν μπορείτε να υπολογίσετε με απόλυτη ακρίβεια αυτές τις πιθανότητες, προσπαθήστε έστω να τις εκτιμήσετε με την μέθοδο Monte Carlo. (Όμως έτσι μπορείτε να διεκδικήσετε μέχρι το 50% των μονάδων αυτής της άσκησης.)

(Μονάδες: 0.5)

γ) Υπολογίστε τις πιθανότητες που ζητούνται στο α) για την περίπτωση που η εκτίμηση δίνεται ως ο μέσος όρος δύο μετρητών Morris, με $\alpha=2$, και με περιορισμό στα 4 bits για κάθε έναν από τους εσωτερικούς μετρητές. (Δηλαδή η εσωτερική μεταβλητή C σε κάθε έναν μετρητή μπορεί να φτάσει μόνο μέχρι την τιμή 15.)

(Μονάδες: 0.5)

Ενότητα Β: Καταμέτρηση διακεκριμένων στοιχείων (Μονάδες: 4.5)

Άσκηση 1

α) Ο σκοπός είναι να υλοποιήσουμε τον αλγόριθμο προσεγγιστικής καταμέτρησης διακεκριμένων στοιχείων που φαίνεται στον ακόλουθο ψευδοκώδικα:

```
integer R ← 0
let h: [0,...,2d-1] → [0,...,2d-1] be a hash function
void insert(integer x)
{
    let r be the number of trailing zeroes of h(x)
    if (r ≥ R)
    {
        R ← r
    }
}

integer query()
{
    return 2R
}
```

Για αρχή, θα δημιουργήσουμε μία ροή 1.000.000 εντελώς τυχαίων στοιχείων με $d=20$, και δεν θα χρησιμοποιήσουμε καμία συνάρτηση κατακερματισμού (ή, αν θέλετε, η συνάρτηση κατακερματισμού h θα είναι η $h(x) = x$).

Το πρόγραμμά σας θα εκτυπώνει μετά από κάθε παραγωγή νέου στοιχείου: 1) το πόσα στοιχεία έχουν παραχθεί μέχρι τότε, 2) το πραγματικό πλήθος διακεκριμένων στοιχείων μέχρι τότε, και 3) την προσέγγιση που δίνει ο αλγόριθμος που περιγράψαμε.

Για παράδειγμα, την στιγμή που θα έχουν παραχθεί 100.000 στοιχεία, το πρόγραμμα θα εκτυπώνει π.χ. την γραμμή: 100000 63212 65536.

Προκειμένου να υλοποιηθεί αποδοτικά το 2) θα πρέπει είτε να χρησιμοποιήσετε κάποια έτοιμη δομή δεδομένων, είτε να υλοποιήσετε εσείς μία (π.χ., προτείνουμε να αποθηκεύετε τα διακεκριμένα στοιχεία σε ένα trie). Η επιτυχής υλοποίηση του 2) πιάνει συνολικά 0.25 μονάδες, αλλά οι 0.125 από αυτές δίνονται μόνο αν υλοποιήσετε εσείς την δική σας δομή δεδομένων.

(Μονάδες: 0.5)

β) Εδώ θα κάνουμε ό,τι και στο α), μόνο που θα αλλάξουμε την κατανομή εισόδου, και επίσης θα χρησιμοποιήσουμε μία (τυχαία επιλεγμένη) συνάρτηση κατακερματισμού.

Συγκεκριμένα, τώρα θα παράγουμε 1.000.000 αριθμούς κατασκευάζοντας συμβολοσειρές των 20 bits ως εξής. Κάθε ένα από τα 5 πρώτα bits είναι 1 με πιθανότητα $1/2$. Κάθε ένα από τα bits 6 μέχρι και 10 είναι 1 με πιθανότητα $1/4$. Κάθε ένα από τα bits 11 μέχρι και 15 είναι 1 με πιθανότητα $1/8$. Και κάθε ένα από τα bits 16 μέχρι και 20 είναι 1 με πιθανότητα $1/16$.

Πρώτα κάντε ό,τι και στο α) (με αυτήν εδώ την κατανομή εισόδου). Ίσως παρατηρείτε ότι η προσέγγιση του πλήθους διακεκριμένων στοιχείων δεν είναι καθόλου καλή. Δώστε μία εξήγηση για αυτό.

Έπειτα επαναλάβετε αυτήν την διαδικασία, χρησιμοποιώντας μία συνάρτηση κατακερματισμού h που ορίζεται ως εξής. Έστω $p = 1.048.583$. (Ο p είναι ο μικρότερος πρώτος αριθμός που είναι μεγαλύτερος από το 2^{20} .) Επιλέξτε τυχαία έναν αριθμό α μεταξύ 1 και $p - 1$, και έναν αριθμό β μεταξύ 0 και $p - 1$. Τότε, το $h(x)$ ορίζεται ως $h(x) = ((\alpha x + \beta) \bmod p) \bmod 2^{20}$.

(Μονάδες: 0.5)

Άσκηση 2

Ο αλγόριθμος που περιγράψαμε στην προηγούμενη άσκηση αναλύεται στην εργασία “The Space Complexity of Approximating the Frequency Moments”, των Alon, Matias, και Szegedy (Journal of Computer and System Sciences 58, 137-147 (1999)).

Εκεί κατοχυρώνονται κάποιες πιθανοτικές εγγυήσεις για τον αλγόριθμο, δεδομένου ότι η συνάρτηση κατακερματισμού που επιλέχτηκε ήταν προϊόν δειγματοληψίας (με ομοιόμορφη κατανομή πιθανότητας) από μία οικογένεια συναρτήσεων κατακερματισμού με κάποιες ειδικές ιδιότητες. Συγκεκριμένα, για την ορθότητα της ανάλυσης απαιτείται να έχουμε:

- 1) $P(\text{tr}(h(x)) \geq r) = 1/2^r$, για κάθε $x \in \{0, 1, \dots, 2^d - 1\}$, και κάθε $r \in \{0, 1, \dots, d\}$.
- 2) $P(\text{tr}(h(x)) \geq r \text{ και } \text{tr}(h(y)) \geq r) = 1/2^{2r}$, για κάθε $x, y \in \{0, 1, \dots, 2^d - 1\}$ με $x \neq y$, και κάθε $r \in \{0, 1, \dots, d\}$.

(Με $\text{tr}(z)$ συμβολίζουμε το πλήθος των τερματικών μηδενικών μιας συμβολοσειράς z .)

α) Η οικογένεια συναρτήσεων κατακερματισμού που προτείναμε στην Άσκηση 1 για $d=20$ (δηλαδή το σύνολο των συναρτήσεων της μορφής $((\alpha x + \beta) \bmod p) \bmod 2^{20}$, για όλα τα πιθανά $\alpha \in \{1, 2, \dots, p - 1\}$, και $\beta \in \{0, 1, \dots, p - 1\}$, όπου $p = 1.048.583$) ικανοποιεί τις ιδιότητες 1) και 2); Αν όχι, θεωρείτε ότι “είναι κοντά” στο να ικανοποιεί αυτές τις ιδιότητες;

(Μονάδες: 0.25)

β) Έστω ότι $V(2^d)$ είναι η οικογένεια συναρτήσεων κατακερματισμού που περιέχει όλες τις συναρτήσεις της μορφής $h : \{0, 1, \dots, 2^d - 1\} \rightarrow \{0, 1, \dots, 2^d - 1\}$ που δίνονται από τον τύπο:

$$h(x) = [(\alpha_{1,1}x_1 + \alpha_{1,2}x_2 + \dots + \alpha_{1,d}x_d + \beta_1) \bmod 2][(\alpha_{2,1}x_1 + \alpha_{2,2}x_2 + \dots + \alpha_{2,d}x_d + \beta_2) \bmod 2] \dots [(\alpha_{d,1}x_1 + \alpha_{d,2}x_2 + \dots + \alpha_{d,d}x_d + \beta_d) \bmod 2],$$
 όπου τα $\alpha_{i,j}$ και β_k είναι 0 ή 1, και $x = x_1x_2\dots x_d$ είναι η συμβολοσειρά εισόδου.

Για παράδειγμα, αν $d=3$, και αν επιλέξουμε $\alpha_{1,1}=0$, $\alpha_{1,2}=0$, $\alpha_{1,3}=0$, $\beta_1=1$, $\alpha_{2,1}=0$, $\alpha_{2,2}=1$, $\alpha_{2,3}=0$, $\beta_2=0$, $\alpha_{3,1}=1$, $\alpha_{3,2}=1$, $\alpha_{3,3}=0$, $\beta_3=0$, τότε π.χ. η συμβολοσειρά 000 απεικονίζεται στην 100, η 001 απεικονίζεται στην 100, η 010 απεικονίζεται στην 111, κ.ο.κ.

(Παρατηρήστε ότι η $V(2^d)$ περιέχει ακριβώς $(2^{d+1})^d$ συναρτήσεις.)

Αποδείξτε ότι η $V(2^d)$ ικανοποιεί τις ιδιότητες 1) και 2) που αναφέραμε στο α).

Υπόδειξη: Εστιάστε για αρχή στο σύνολο S των συναρτήσεων $h : \{0, 1, \dots, 2^d - 1\} \rightarrow \{0, 1\}$ που περιέχει όλες τις συναρτήσεις της μορφής $h(x) = (\alpha_1 x_1 + \alpha_2 x_2 + \dots + \alpha_d x_d + \beta) \bmod 2$, όπου τα α_i και το β είναι 0 ή 1. Αποδείξτε ότι, για κάθε συμβολοσειρά x των d bits, ακριβώς οι μισές συναρτήσεις στο S απεικονίζουν το x στο 0. Έπειτα, δείξτε ότι, για κάθε δύο διακεκριμένες συμβολοσειρές x και y των d bits, ακριβώς το $1/4$ των συναρτήσεων εντός του S απεικονίζουν και το x και το y στο 0. Τέλος, αξιοποιήστε αυτές τις πληροφορίες για να αποδείξετε το ζητούμενο.

(Μονάδες: 0.5)

γ) Μπορούμε να γενικεύσουμε την οικογένεια συναρτήσεων κατακερματισμού που θεωρήσαμε στην Άσκηση 1 ως εξής. Έστω p ο μικρότερος πρώτος αριθμός που είναι μεγαλύτερος του 2^d . Τότε, θεωρούμε την οικογένεια $H(2^d)$ όλων των συναρτήσεων της μορφής $h(x) = ((\alpha x + \beta) \bmod p) \bmod 2^d$, για όλα τα πιθανά $\alpha \in \{1, 2, \dots, p-1\}$ και $\beta \in \{0, 1, \dots, p-1\}$.

Ποια θεωρείτε ότι είναι τα πλεονεκτήματα και τα μειονεκτήματα της χρήσης των οικογενειών συναρτήσεων κατακερματισμού $V(2^d)$ και $H(2^d)$ για τον αλγόριθμο εκτίμησης πλήθους διακεκριμένων στοιχείων της Άσκησης 1; (Διακρίνετε δύο περιπτώσεις, όπου το d είναι σχετικά μικρό, π.χ. το πολύ 20, ή αρκετά μεγαλύτερο, όπως π.χ. 100.)

(Μονάδες: 0.25)

Άσκηση 3

α) Εδώ θα υλοποιήσουμε έναν διαφορετικό αλγόριθμο για την προσεγγιστική καταμέτρηση διακεκριμένων στοιχείων, που δίνει καλύτερες εγγυήσεις από αυτόν που περιγράψαμε στην Άσκηση 1.

Συγκεκριμένα, θα εφαρμόσουμε τον αλγόριθμο 1 που περιγράφεται στην εργασία “Counting distinct elements in a data stream” των Bar-Yossef, Jayram, Kumar, Sivakumar, και Trevisan (BJKST), RANDOM, 2002. Ο αλγόριθμος αυτός λαμβάνει ως είσοδο ένα ϵ , και δίνει μία εκτίμηση N του πλήθους F_0 των διακεκριμένων στοιχείων σε μια ροή στοιχείων, η οποία ικανοποιεί την ανισότητα $(1 - \epsilon)F_0 \leq N \leq (1 + \epsilon)F_0$, με πιθανότητα τουλάχιστον $2/3 - 1/m$, χρησιμοποιώντας χώρο $O((1/\epsilon^2)\log(m))$, όπου m είναι ο μέγιστος αριθμός που μπορεί να εμφανιστεί στην ροή. (Αρχικά εξηγήστε από ποιες απόψεις αυτές οι εγγυήσεις είναι καλύτερες από εκείνες που γνωρίζετε για τον αλγόριθμο που περιγράψαμε στην Άσκηση 1.)

Σε αυτήν την άσκηση, ο σκοπός είναι να εφαρμόσετε τον BJKST αλγόριθμο στις δύο κατανομές εισόδου που περιγράφονται στα α) και β) της Άσκησης 1, και θα εργαστείτε ακριβώς όπως και σε εκείνες τις ασκήσεις. Απλώς, εφόσον ο BJKST υποθέτει ότι η είσοδός του είναι ακέραιοι, θα χρειαστεί να κάνουμε την εξής μικρή αλλαγή στην μορφή της εισόδου: κάθε συμβολοσειρά x που εμφανίζεται στην ροή, θα μετατρέπεται στον αντίστοιχο αριθμό μεταξύ 0 και $2^{20} - 1$ (υποθέτουμε δηλαδή ότι το x είναι η αναπαράσταση ενός αριθμού στο δυαδικό σύστημα αρίθμησης), και έπειτα θα του προστίθεται το 1. Έτσι θα λαμβάνουμε τα στοιχεία εισόδου.

Ο αλγόριθμος BJKST περιγράφεται από τα εξής βήματα.

1) Επιλέγουμε μία συνάρτηση κατακερματισμού $h : [1, \dots, m] \rightarrow [1, \dots, M]$, όπου $M = m^3$. (Για εμάς ισχύει ότι $m = 2^{20}$.)

2) Θέτουμε $t = \lceil 96/\varepsilon^2 \rceil$.

3) Για κάθε x που εμφανίζεται στην ροή, υπολογίζουμε την τιμή $h(x)$. Διατηρούμε (με μια αποδοτική δομή δεδομένων) τις t μικρότερες διακεκριμένες τιμές που συναντήσαμε, καθόλη την διάρκεια της ροής.

4) Ως εκτίμηση του πλήθους διακεκριμένων στοιχείων, επιστρέφουμε την τιμή $N = tM/v$, όπου v είναι η t -οστή μικρότερη διακεκριμένη τιμή που έχουμε αποθηκεύσει. (Ας αρχίσουμε να εκτυπώνουμε το N μονάχα απ' την στιγμή που θα έχουμε συγκεντρώσει τουλάχιστον t διακεκριμένες τιμές.)

Η συνάρτηση κατακερματισμού θα έχει την ίδια μορφή όπως και στο β) της Άσκησης 1, μόνο που το p εδώ θα είναι το q^3 , όπου $q = 1.048.583$ είναι ο μικρότερος πρώτος αριθμός που είναι μεγαλύτερος από το 2^{20} . (Τα α και β θα επιλέγονται τυχαία μεταξύ 1 και $p - 1$, και 0 και $p - 1$, αντίστοιχα.) Ως αποτέλεσμα $h(x)$ θα επιστρέφεται η τιμή $((\alpha x + \beta) \bmod p) \bmod M + 1$.

Υλοποιήστε τον αλγόριθμο για $\varepsilon=0.1$ και $\varepsilon=0.05$, και τρέξτε τον στις κατανομές εισόδου που περιγράψαμε.

(Μονάδες: 2)

β) Βελτιώστε την πιθανοτική εγγύηση του αλγορίθμου, εφαρμόζοντας την τεχνική του διαμέσου. Συγκεκριμένα, χρησιμοποιήστε έναν κατάλληλο αριθμό επαναλήψεων, ώστε, ακολουθώντας μια θεωρητική ανάλυση, να κάνετε την πιθανότητα επιτυχίας τουλάχιστον 99.9%.

(Μονάδες: 0.5)

Ενότητα Γ: Φίλτρα Bloom (Μονάδες: 5)

Άσκηση 1: Υπολογισμός τομής

Έστω ότι έχουμε μία συλλογή από τεράστια αρχεία που βρίσκονται κατανεμημένα σε δύο servers A και B. Ορισμένα αρχεία περιέχονται και στους δύο servers (ως ακριβή αντίγραφα), και θα θέλαμε να τα βρούμε. Μας δίνεται ως πληροφορία ότι αυτά τα κοινά αρχεία είναι πολύ λιγότερα απ' όσα περιέχονται στους A και B. Η επικοινωνία μεταξύ των servers κοστίζει πολύ, οπότε θα θέλαμε να υπολογίσουμε τα κοινά αρχεία των A και B ελαχιστοποιώντας την επικοινωνία μεταξύ τους. Για τον σκοπό αυτό, μπορούμε να χρησιμοποιήσουμε φίλτρα Bloom ως εξής.

Ο A θα υπολογίσει ένα φίλτρο Bloom για τα αρχεία που περιέχονται σε αυτόν, και θα το στείλει στον B για να ελέγξει ποια από τα δικά του βρίσκονται στον A. Έπειτα, ο B θα στείλει στον A αυτά που πέρασαν το φίλτρο, και ο A θα ελέγξει ποια από αυτά ανήκουν όντως σε αυτόν. (Υποθέτουμε ότι ο A μπορεί να κάνει γρήγορο έλεγχο ύπαρξης, έχοντας οργανώσει τα αρχεία του σε μια αποδοτική δομή δεδομένων, π.χ., σε tries, ή, έχοντας ταξινομήσει τα αρχεία του, κάνοντας δυαδική αναζήτηση.)

Ας πούμε ότι η παραπάνω διαδικασία είναι “ένας γύρος επικοινωνίας”. Προφανώς κάποια αρχεία που ο B έστειλε στον A δεν ανήκουν στον A, αλλά στάλθηκαν λόγω του false positive bias του φίλτρου Bloom. Παρά ταύτα, κατά πάσα πιθανότητα πετύχαμε κάτι καλύτερο απ' το να στέλναμε όλα τα αρχεία του B στον A.

Υπάρχει όμως το ενδεχόμενο να μπορούμε να κάνουμε και κάτι πολύ καλύτερο. Αντί ο B στον πρώτο γύρο να στείλει τα αρχεία του, θα στείλει ένα δικό του φίλτρο Bloom στον A, που σχηματίζεται από τα αρχεία του B που πέρασαν από το φίλτρο του A. Έπειτα ο A θα στείλει ένα καινούριο φίλτρο στον B με βάση αυτό που του έστειλε ο B, και τώρα ο B θα στείλει στον A όσα αρχεία περνούν από αυτό το δεύτερο φίλτρο, όπως και πριν. Ας πούμε ότι αυτή η διαδικασία συνιστά “δύο γύρους επικοινωνίας”.

Αυτό το σχήμα επικοινωνίας γενικεύεται άμεσα σε “k γύρους επικοινωνίας”.

α) Υλοποιήστε την παραπάνω ιδέα ως εξής.

Πρώτα απ' όλα, θα φροντίσουμε οι A και B να περιέχουν 100.000 αρχεία ο καθένας, αλλά μόνο 10 θα είναι κοινά σε αυτούς. Τα “αρχεία” για εμάς θα είναι απλώς συμβολοσειρές των 100 bits.

Για να υλοποιήσουμε αυτήν την κατάσταση, πρώτα θα φτιάξουμε 10 τυχαία αρχεία. (Δηλαδή, 10 εντελώς τυχαίες συμβολοσειρές των 100 bits.) Έπειτα θα φτιάξουμε 99.990 τυχαία αρχεία για τον A, 99.990 τυχαία αρχεία για τον B, και θα προσθέσουμε στους A και B επιπλέον τα 10 αρχεία που φτιάξαμε στην αρχή.

Ισχυριζόμαστε ότι αυτή η διαδικασία εξασφαλίζει με πρακτικά απόλυτη βεβαιότητα ότι μόνο τα 10 αρχεία που φτιάξαμε στην αρχή θα ανήκουν στους A και B ταυτόχρονα. Τεκμηριώστε αυτόν τον ισχυρισμό, εκτιμώντας την πιθανότητα τα κοινά αρχεία στους A και B, μετά από την όλη αυτή διαδικασία, να είναι πάνω από 10.

Κάθε συνάρτηση κατακερματισμού που θα χρησιμοποιήσουμε για τα φίλτρα Bloom κατασκευάζεται ως εξής. Έστω $p = 1048583$, και έστω N το πλήθος των bits του φίλτρου Bloom. (Ο p είναι ο μικρότερος πρώτος αριθμός που είναι μεγαλύτερος από το 2^{20} .) Αρχικά, επιλέγουμε τυχαία έναν αριθμό α μεταξύ 1 και $p - 1$, και έναν τυχαίο αριθμό β μεταξύ 0 και $p - 1$. Αυτές οι τέσσερις παράμετροι (α , β , p , N) καθορίζουν μια συνάρτηση κατακερματισμού h . Έστω τώρα ένα αρχείο X , και ο σκοπός είναι να περιγράψουμε πώς υπολογίζεται το $h(X)$. Αρχικά, χωρίζουμε το X σε 5 συνεχόμενα κομμάτια B_1, B_2, B_3, B_4, B_5 των 20 bits. Θεωρούμε κάθε ένα από αυτά τα κομμάτια ως έναν αριθμό μεταξύ 0 και $2^{20} - 1$, που η δυαδική του αναπαράσταση δίνεται από τα bits που συνιστούν το κομμάτι. (Π.χ., ένα κομμάτι 000000000000000001101 ερμηνεύεται ως ο αριθμός 13.) Αρχικοποιούμε μια τιμή $c_0 \leftarrow 0$, και υπολογίζουμε αναδρομικά την τιμή $c_i = (\alpha(B_i + c_{i-1}) + \beta) \bmod p$, για κάθε $i \in \{1, 2, 3, 4, 5\}$. Τελικά, επιστρέφουμε το $c_5 \bmod N$.

Για αρχή, ας υποθέσουμε ότι $N = 500.000$. (Οπότε, κάθε φίλτρο Bloom που στέλνεται από τον A στον B, ή αντίστροφα, αποτελείται από 500.000 bits. Παρατηρήστε όμως ότι ο κάθε ένας από τους A και B θα πρέπει να στέλνει στον άλλον και τις συναρτήσεις κατακερματισμού που χρησιμοποίησε.)

Υλοποιήστε την μέθοδο που περιγράψαμε για τον υπολογισμό των κοινών αρχείων των A και B. Το πρόγραμμά σας αρκεί να εκτυπώνει μονάχα το πλήθος των positives που υπολόγισε ο B στο τελευταίο φίλτρο Bloom που του έστειλε ο A. Ας ονομάσουμε αυτήν την παράμετρο ύψιστου ενδιαφέροντος **count**. (Όμως ελέγξτε κι όλες ότι στα αρχεία που ο B πήρε positive στο τέλος περιλαμβάνονται τα 10 που ανήκουν και στον A, και εκτυπώστε “true” σε αυτήν την περίπτωση.)

Η υλοποίησή σας θα πρέπει να αφήνει την ελευθερία στον χρήστη να επιλέξει: 1) το πλήθος των συναρτήσεων κατακερματισμού (που θα αφορά όλα τα φίλτρα Bloom που θα δημιουργηθούν), και 2) το πλήθος των γύρων επικοινωνίας.

Επιπλέον, δώστε σαν επιλογή, αντί όλα τα φίλτρα Bloom να αποτελούνται από 500.000 bits, το πλήθος των bits να αλλάζει με βάση το πόσα αρχεία εκτίμησε ο ένας να περιέχονται στον άλλον. Συγκεκριμένα, αν π.χ. πρόκειται να έχουμε πάνω από έναν γύρο επικοινωνίας, τότε ο B, αν στον πρώτο γύρο επικοινωνίας βρει ότι N_1 αρχεία του περνούν το φίλτρο που του έστειλε ο A, τότε θα στείλει ένα φίλτρο Bloom στον A με $5N_1$ bits. Έπειτα, αν ο A βρει N_2 αρχεία του να περνούν το φίλτρο που του έστειλε το B, τότε στέλνει ένα φίλτρο Bloom στον B με $5N_2$ bits, κ.ο.κ. Θεωρείτε ότι αυτό θα βελτιώσει ή θα χειροτερέψει το αποτέλεσμα **count**; (Για το πρόβλημά μας, προφανώς το “χειροτέρεμα” είναι το **count** να αυξάνεται.)

Δοκιμάστε τις εξής παραλλαγές (σε όλους τους δυνατούς συνδυασμούς): i) το πλήθος των συναρτήσεων κατακερματισμού να είναι 1, 2, 3, 5, 10, και ii) το πλήθος των γύρων επικοινωνίας να είναι 1, 2, 3, 4, 5. Για ποιον συνδυασμό βλέπετε να έχουμε το βέλτιστο **count**;

(Μονάδες: 1.75)

β) Ίσως παρατηρήσατε (ή υποψιάζεστε) ότι όσο το πλήθος των γύρων επικοινωνίας αυξάνεται, τόσο περισσότερο αναμένεται να βελτιώνεται (δηλαδή να μειώνεται) το

πλήθος των αρχείων που πρέπει τελικά ο B να στείλει στον A (δηλαδή το **count**). Δώστε μία πειστική μαθηματική εξήγηση για αυτήν την παρατήρηση (ή υποψία), ή καταρρίψτε αυτόν τον ισχυρισμό.

(Μονάδες: 0.25)

Άσκηση 2: Ιχνηλάτηση πακέτου (Packet Tracing)

Έστω ότι έχουμε ένα δίκτυο από routerάκια, τα οποία προωθούν μηνύματα το ένα στο άλλο. Ανά πάσα ώρα και στιγμή, ένα routerάκι μπορεί να δημιουργήσει ένα μήνυμα, το οποίο θα το στείλει σε κάποιο από τα routerάκια με τα οποία είναι συνδεδεμένο ως πομπός. (Το ποιος μπορεί να στείλει μήνυμα σε ποιον εξαρτάται από μία προκαθορισμένη τοπολογία.) Ένα routerάκι που δέχεται μήνυμα, μπορεί να αναρωτιέται ποιος ήταν αυτός που το δημιούργησε αρχικά. Σε αυτήν την περίπτωση, μπορεί να ρωτήσει ένα-ένα τα routerάκια τα οποία συνδέονται με αυτό (ως πομποί του), ώστε να μάθει ποιο του έστειλε το μήνυμα, και έπειτα αυτό να ρωτήσει με την σειρά του τους πομπούς του. Το πρόβλημα είναι ότι τα routerάκια έχουν πολύ περιορισμένη μνήμη, οπότε δεν μπορούν να κρατούν αυτούσια τα μηνύματα που δέχονται.

Σε ένα τέτοιο σενάριο, τα φίλτρα Bloom μπορούν να φανούν πολύ χρήσιμα. Υποθέτουμε ότι κάθε routerάκι κρατάει ένα φίλτρο Bloom για τα μηνύματα που δέχτηκε, και άρα η ιχνηλάτηση του μηνύματος θα γίνει χρησιμοποιώντας τα φίλτρα Bloom που είναι αποθηκευμένα στα routerάκια. Παρατηρήστε ότι αυτό θα κάνει ορισμένα routerάκια να ισχυρίζονται ότι ένα μήνυμα πέρασε από αυτά, ή δημιουργήθηκε από αυτά, χωρίς να ισχύει κάτι τέτοιο. Αν όμως καταλήξουμε σε μοναδικό routerάκι που ισχυρίζεται ότι δημιούργησε το μήνυμα, τότε θεωρούμε την ιχνηλάτηση πετυχημένη.

α) Ο σκοπός είναι να προσομοιώσουμε την διαδικασία προώθησης μηνυμάτων και ιχνηλάτησής τους, χρησιμοποιώντας φίλτρα Bloom, και να υπολογίσουμε το πλήθος των πετυχημένων ιχνηλατήσεων.

Συγκεκριμένα, έχουμε διάφορα δίκτυα από routerάκια, τα οποία έχουν την μορφή κατευθυνόμενου γραφήματος. Τα routerάκια είναι αριθμημένα ως 1, 2, 3, Υποθέτουμε ότι μόνο τα 1, 2, . . . , 10 δημιουργούν μηνύματα, και το τελευταίο router (δηλαδή αυτό με το μεγαλύτερο νούμερο) είναι ο παραλήπτης των μηνυμάτων που εκκινεί την διαδικασία ιχνηλάτησης.

Τα γραφήματα περιέχονται στον φάκελο Networks, στην σελίδα του μαθήματος στο ecourse, και δίνονται σε μορφή αρχείου κειμένου που έχει την εξής μορφή. Η πρώτη γραμμή έχει την μορφή “ $n\ m$ ”, όπου n είναι το πλήθος των κόμβων (δηλαδή των router), και m είναι το πλήθος των ακμών (συνδέσεων). Κάθε μια από τις επόμενες m γραμμές έχει την μορφή “ $x\ y$ ”, που σημαίνει ότι ο x μπορεί να στέλνει μηνύματα στον y . Τα γραφήματα αυτά δεν περιέχουν κύκλους (οπότε η προώθηση μηνύματος δεν πρόκειται να πέσει σε ατέρμονα βρόχο), και από κάθε έναν από τους κόμβους 1, 2, . . . , 10 υπάρχει (κατευθυνόμενο) μονοπάτι προς τον n (τον τελικό κόμβο-παραλήπτη). Επιπλέον, τα routerάκια 1, 2, . . . , 10 έχουν μόνο εξερχόμενες συνδέσεις προς άλλα routerάκια (δηλαδή λειτουργούν μονάχα ως πομποί, και όχι ως δέκτες).

Η διαδικασία δημιουργίας και προώθησης μηνύματος λειτουργεί ως εξής. Επιλέγουμε να παραχθούν 100.000 μηνύματα, όπου κάθε μήνυμα συνιστά μια συμβολοσειρά των 100 bits. Κάθε μήνυμα παράγεται τυχαία, και με τυχαίο τρόπο επιλέγεται ένα από τα routerάκια 1, 2, . . . , 10 ως αυτό που το δημιουργήσει. Έπειτα, δεδομένου ότι το μήνυμα βρίσκεται σε ένα router x που δεν είναι το τερματικό (δηλαδή το n), επιλέγουμε τυχαία ένα από τα router-δέκτες του x (με άλλα λόγια: έναν κόμβο y για τον οποίο υπάρχει ακμή (x,y)), και στέλνουμε το μήνυμα στο y . Κάθε router έχει το δικό του φίλτρο Bloom, και κάθε φορά που περνάει μήνυμα από αυτό (ή που δημιουργείται από αυτό), το φίλτρο Bloom του καταγράφει ότι πέρασε από εκεί αυτό το μήνυμα.

Μόλις σταλθούν και τα 100.000 μηνύματα, το τελευταίο routerάκι αρχίζει να τα ιχνηλατεί ένα προς ένα. Για κάθε μήνυμα, ρωτάει τους πομπούς του αν το δέχτηκαν. Όσοι είπαν “NAI”, κάνουν το ίδιο, μέχρι κάποιο από τα αρχικά routerάκια (δηλαδή κάποιο από τα 1, 2, . . . , 10) να πει “NAI”. Σε περίπτωση που μόνο ένα από τα αρχικά routerάκια καταλήξει να έχει πει “NAI”, θεωρούμε την ιχνηλάτηση πετυχημένη.

Στο τέλος, θα πρέπει να εκτυπώνουμε το πλήθος των πετυχημένων ιχνηλατήσεων. (Προφανώς αυτό θα ποικίλλει, ανάλογα με την τυχειότητα στα φίλτρα Bloom, αλλά αναμένεται ίσως να υπάρχει κάποια ξεκάθαρη τιμή για κάθε γράφημα, γύρω απ’ την οποία συγκεντρώνονται οι τιμές από κάθε πείραμα ιχνηλάτησης.)

Θα δώσουμε σε κάθε routerάκι 10.000 bits μνήμης για το φίλτρο Bloom του, και για αρχή ας δοκιμάσουμε μόνο μία συνάρτηση κατακερματισμού (μία διαφορετική, για κάθε φίλτράκι). Οι συναρτήσεις κατακερματισμού θα είναι ακριβώς του ίδιου τύπου όπως και στην Άσκηση 1. Έπειτα δοκιμάστε περισσότερες συναρτήσεις κατακερματισμού (για κάθε φίλτράκι), και αναφέρετε για κάθε γράφημα ποιο πλήθος συναρτήσεων κατακερματισμού σάς φαίνεται ότι δίνει την καλύτερη απόδοση. (Η απόδοση εδώ μετράται ως το αναμενόμενο πλήθος των επιτυχημένων ιχνηλατήσεων.)

Ίσως μπορείτε να αντιληφθείτε ότι, κατά την διαδικασία της ιχνηλάτησης, ορισμένα routerάκια μπορεί να δεχτούν πάρα πολλά requests για έλεγχο ύπαρξης του μηνύματος στο φίλτρο Bloom τους. (Εξηγήστε γιατί μπορεί να συμβεί αυτό, και δώστε και ένα θεωρητικό παράδειγμα.) Αυτό θα ήταν αναπόφευκτο σε μια πραγματική εφαρμογή, και θα πρέπει να κρατήσετε αυτήν την υλοποίηση.

Παρά ταύτα, στο πλαίσιο της προσομοίωσης, μπορείτε να υλοποιήσετε επιπλέον και έναν πολύ πιο αποδοτικό αλγόριθμο για τον υπολογισμό των πετυχημένων ιχνηλατήσεων.

(Μονάδες: 2.5)

β) Ίσως παρατηρήσατε ότι σε ορισμένα γραφήματα η ιχνηλάτηση είναι γενικά πιο πετυχημένη απ’ ό,τι σε κάποια άλλα. Πώς εξηγείτε την διαφορά στην επίδοση μεταξύ των διαφόρων γραφημάτων; Στην επόμενη σελίδα μπορείτε να δείτε μία αναπαράσταση όλων των γραφημάτων που βρίσκονται στον φάκελο Networks.

(Μονάδες: 0.5)

